

Software Design Specification

통합 게임 데이터 및 하드웨어 최적화 플랫폼

2021663046 이건우

Chapter 1

System Overview

시스템 개요, 아키텍처, 모듈 및 개념적 설계

1.1. 시스템 개요

- ▶ **다면 웹 서비스 플랫폼:** 분산된 게임 데이터를 통합하고 사용자 하드웨어 제원을 분석하여 최적화된 그래픽 설정값을 제공합니다.
- ▶ **외부 계정 연동:** OAuth 2.0 프로토콜을 활용한 외부 플랫폼 (Steam, Riot) API 연동 및 라이브러리 데이터를 동기화합니다.
- ▶ **통합 게임 대시보드:** Recharts를 활용하여 플레이 타임 누적 추이 및 업적 달성 데이터를 시각화합니다.
- ▶ **유사도 기반 매칭:** CPU(30%), GPU(50%), RAM(10%), 해상도(10%) 가중치 모델을 통해 90% 이상 일치하는 세팅을 추천합니다.



1.2 & 1.3 아키텍처 및 모듈



USER MODULE

Frontend Tier

React.js 및 Tailwind CSS로 구성. SPA 형태로 사용자 상호작용 및 데이터 시각화를 전담합니다. 다수의 하드웨어 프로필(CRUD) 관리를 포함합니다.



POST MODULE

Application Tier

Node.js/Express 환경의 Stateless 서버. 비즈니스 로직, 외부 API 통신 및 Hardware Matching Engine(유사도 연산)을 통해 최적 프로파일을 매칭합니다.



AUTH MODULE

Data Tier & Security

PostgreSQL과 Redis 캐시를 활용. OAuth 2.0 연동 및 AES-256 토큰 암호화, JWT 기반 클라이언트 세션 발급을 처리합니다.

1.4 & 1.5 데이터 및 API 설계 원칙

개념적 데이터 설계 (Data Design)

- ▶ **3NF 준수:** 전체 데이터 모델은 제3정규화를 준수하여 데이터 무결성을 보장합니다.
- ▶ **핵심 도메인:** users, hardware_profiles, games, optimization_profiles 4개 영역으로 분리.
- ▶ **식별 및 관계:** 고유 식별자(UUID)를 통한 테이블
- ▶ **경향 유연한 스키마:** 복잡한 인게임 옵션 트리는 settings_json 형태의 Document 스키마로 저장하여 유연성을 확보합니다.

인터페이스 설계 (API Design)

- ▶ **RESTful 아키텍처:** 명사/복수형 자원 표기 및 HTTP Method(GET, POST, PUT, DELETE)를 분리.
- ▶ **JSON 통신:** application/json Payload 및 표준 응답 래퍼(status, message, data) 사용.
- ▶ **보안 인증:** Public Endpoint 외 모든 API는 Authorization: Bearer 헤더 검증 필수.
- ▶ **표준 상태 코드:** 400(검증 실패), 401(인증 실패), 403, 404, 500 등 명확한 에러 코드를 반환합니다.

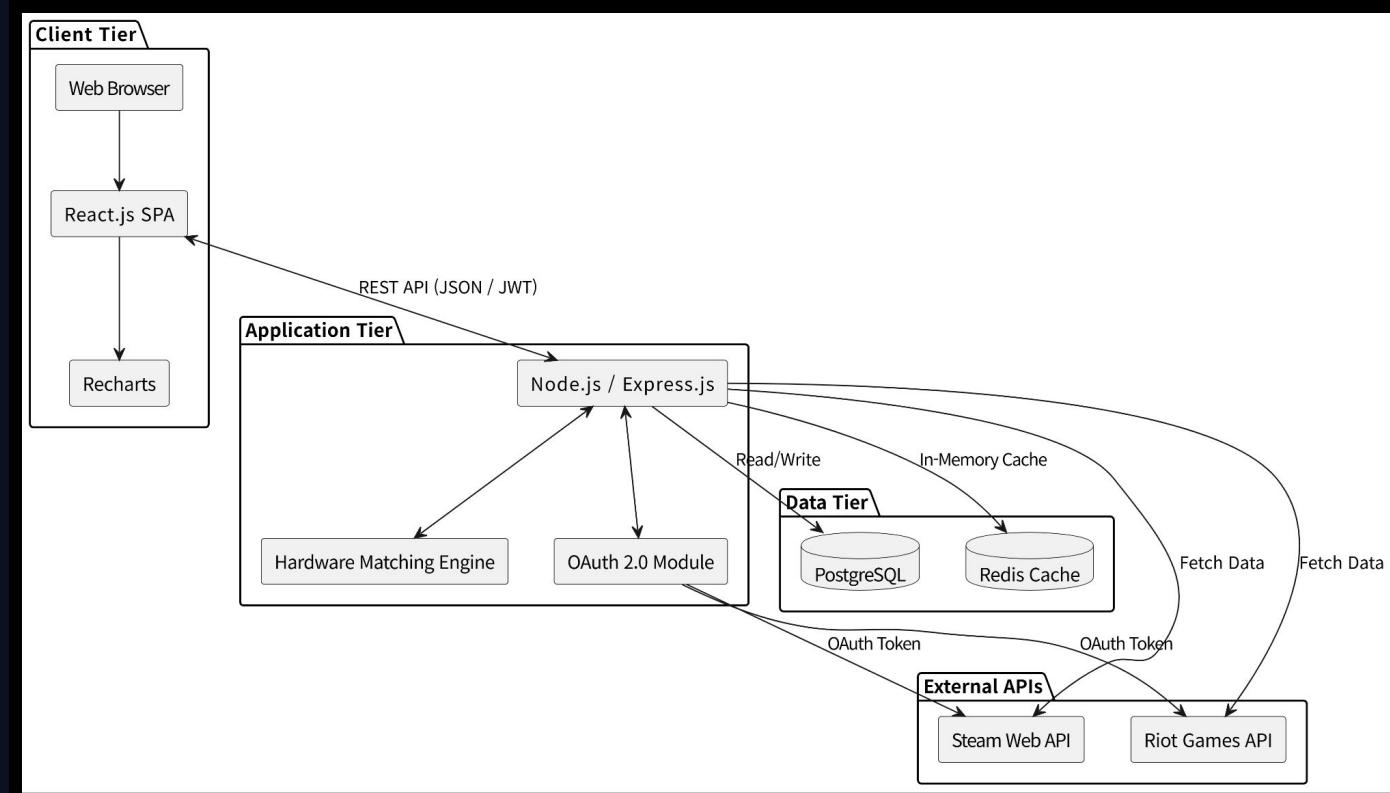
Chapter 2

Architecture Diagram

인프라스트럭처 배포 및 시스템 계층 상세

2. 상세 아키텍처 구성

- ▶ **Client (웹 브라우저):** React.js 기반 SPA로 Tailwind CSS 반응형 UI 및 Recharts 시각화를 제공.
- ▶ **Server (Node.js/Express):** Hardware Matching Engine 및 OAuth 2.0 Module 구동. Stateless 구조로 확장성 확보.
- ▶ **Database & Cache:** PostgreSQL(관계형 영구 데이터), Redis(API Rate Limit 제어 및 단기 캐싱).
- ▶ **외부 API 연동:** Steam Web API 및 Riot Games API를 통한 메타데이터 동기화.
- ▶ **보안 및 배포 환경:** TLS 1.3 암호화, AWS 클라우드 기반 Kubernetes (K8s) 오케스트레이션 및 Docker 컨테이너 배포.



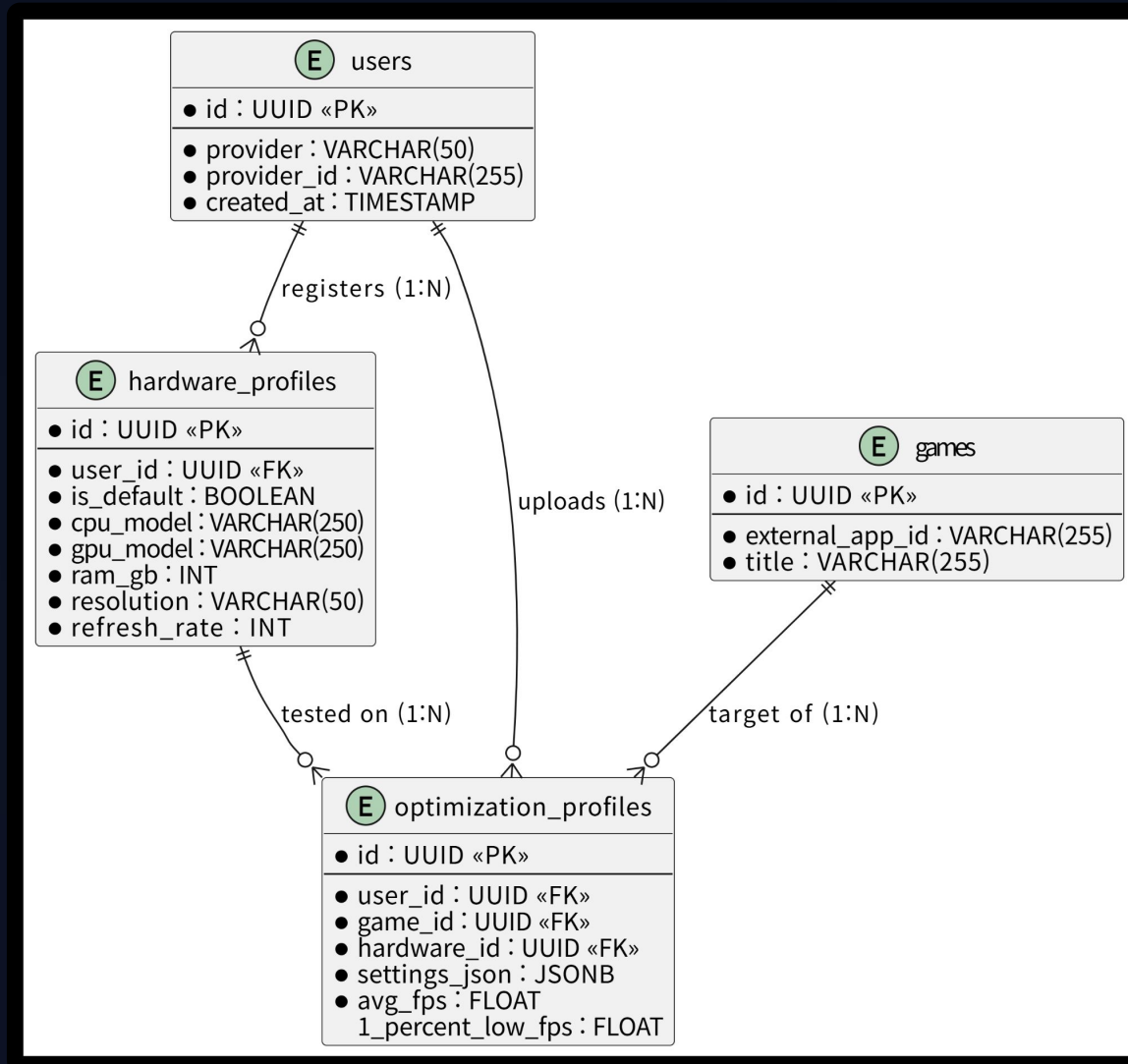
Chapter 3

Entity Relationship Diagram

Detailed Physical Data

Model

3. Detailed Physical Data Model



3. Detailed Physical Data Model

테이블명	컬럼명	Data Type	속성 (PK/FK/NN)	설명
users	id	UUID	PK, NN	사용자 고유 식별자
	provider	VARCHAR(50)	NN	외부 인증 제공자 (steam, riot 등)
	provider_id	VARCHAR(255)	NN	외부 플랫폼의 사용자 식별자
	created_at	TIMESTAMP	NN	시스템 계정 생성 일시
hardware_profiles	id	UUID	PK, NN	하드웨어 프로파일 고유 식별자
	user_id	UUID	FK, NN	소유자 식별자 (users.id)
	cpu_model	VARCHAR(250)	NN	CPU 명칭
	gpu_model	VARCHAR(250)	NN	GPU 명칭
	ram_gb / res / hz	INT / VARCHAR	NN	RAM, 해상도, 주사율 데이터

3. Detailed Physical Data Model

테이블명	컬럼명	Data Type	속성 (PK/FK/NN)	설명
optimization_profiles	id	UUID	PK, NN	최적화 프로파일 식별자
game_id / user_id	UUID	FK, NN	대상 게임 및 작성자 식별자	
hardware_id	UUID	FK, NN	벤치마크 테스트 하드웨어	
settings_json	JSONB	NN	인게임 세부 그래픽 설정 객체	
avg_fps	FLOAT	NN	측정된 평균 프레임 수치	

Chapter 4

API Specification

Detailed Implementation Guide

4. API Specification Details



외부 플랫폼 인증

GET /auth/{provider}/callback

OAuth 2.0 프로바이더로부터
Authorization Code를 받아 토큰 교환 및
세션(JWT)을 생성합니다. 인증이 필요
없는 Public 엔드포인트입니다.



하드웨어 프로필 등록

POST /users/hardware-profiles

사용자의 PC 자원(CPU, GPU, RAM,
해상도 등) Payload를 입력받아 프로필을
생성합니다. Authorization: Bearer 토큰이
필수입니다.



최적화 세팅 추천

GET /profiles/recommendations

클라이언트 하드웨어와 DB를 교차
분석하여 90% 이상 유사한 환경의 최적화
그래픽 설정값 및 예상 FPS를 반환합니다.

Q&A

발표 대한 질문을 받습니다.

Thank you for your attention